CENTRO FP SUPERIOR
CAMPUS CÁMARA COMERCIO SEVILLA

AEE. Bitácora III. Conexiones Empresarialos
Taller Técnico: Operacion Escudo

Manuel Parrilla Lahoz

Unión Europea

Reto A: Auditoría de Accesos e Intrusiones (Log Analysis y Scripting)

El archivo de registro de autenticación es vuestra "caja negra". Si alguien intenta forzar la entrada utilizando técnicas de *Fuerza Bruta* o *Ataques de Diccionario*, el rastro indeleble quedará grabado aquí.

1. **Generación de ataques controlados**. Desde vuestro equipo host (Windows) o desde una segunda máquina virtual, realiza al menos 5 intentos fallidos de conexión SSH contra vuestro servidor (ej: ejecutando *ssh administrador_falso@ip_del_servidor* y metiendo contraseñas inventadas).

   Primero obtenemos la ip del servidor ubuntu al que vamos a atacar:

```
enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group d

  link/ether 08:00:27:03:10:13 brd ff:ff:ff:ff:ff:ff
  inet 192.168.20.33/22 metric 100 brd 192.168.23.255 scope global dynamic enp0s3
```

   iniciamos el ataque desde el host(windows)

```
PS C:\Users\DAW1> ssh administrador_falso@192.168.20.33
administrador_falso@192.168.20.33's password:
Permission denied, please try again.
administrador_falso@192.168.20.33's password:
Permission denied, please try again.
administrador_falso@192.168.20.33's password:
administrador_falso@192.168.20.33: Permission denied (publickey,password).
PS C:\Users\DAW1>
```

2. **Análisis Forense Manual**. Accede a */var/log/auth.log* utilizando herramientas de visualización rápida como *tail -n 50*, *less* o filtrando con *grep*. Debéis ser capaces de identificar y hacer una captura de pantalla donde se vea claramente: la dirección IP de origen del atacante, el puerto efímero utilizado y el mensaje literal de error del sistema (algo parecido a *Failed password for invalid user...*).

```
r 17 12:12:37 daw1 systemd-logind[772]: New session 1 of user daw1.
r 17 12:12:37 daw1 systemd: pam_unix(systemd-user:session): session opened for user daw1 by (uid=

r 17 12:12:45 daw1 sudo:     daw1 : TTY=tty1 ; PWD=/home/daw1 ; USER=root ; COMMAND=/usr/bin/apt
date
r 17 12:12:45 daw1 sudo: pam_unix(sudo:session): session opened for user root by daw1(uid=0)
r 17 12:12:48 daw1 sudo: pam_unix(sudo:session): session closed for user root
r 17 12:15:25 daw1 sshd[9422]: Invalid user administrador_falso from 192.168.23.7 port 52010
r 17 12:15:28 daw1 sshd[9422]: pam_unix(sshd:auth): check pass; user unknown
r 17 12:15:28 daw1 sshd[9422]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
y=ssh ruser= rhost=192.168.23.7
r 17 12:15:30 daw1 sshd[9422]: Failed password for invalid user administrador_falso from 192.168.
7 port 52010 ssh2
r 17 12:15:43 daw1 sshd[9422]: pam_unix(sshd:auth): check pass; user unknown
r 17 12:15:45 daw1 sshd[9422]: Failed password for invalid user administrador_falso from 192.168.
7 port 52010 ssh2
r 17 12:15:47 daw1 sshd[9422]: pam_unix(sshd:auth): check pass; user unknown
r 17 12:15:49 daw1 sshd[9422]: Failed password for invalid user administrador_falso from 192.168.
7 port 52010 ssh2
r 17 12:15:51 daw1 sshd[9422]: Connection reset by invalid user administrador_falso 192.168.23.7
t 52010 [preauth]
r 17 12:15:51 daw1 sshd[9422]: PAM 2 more authentication failures; logname= uid=0 euid=0 tty=ssh
```

3. **Automatización inteligente con Bash**. Como imaginaréis, un administrador no puede pasarse la mañana leyendo miles de líneas de texto plano. Debéis crear un *script* en *bash* en la ruta *scripts/check_intruders.sh* que automatice esta tarea. El script debe:
    - Filtrar exclusivamente los intentos fallidos de conexión de las últimas horas. *Pista*: Herramientas como *grep*, *awk* o *sed* serán vuestras mejores amigas para extraer columnas de texto.
    - Contar matemáticamente el número total de errores detectados.
    - Generar (o sobrescribir) un archivo de reporte limpio llamado **alertas.txt** que incluya un encabezado con la fecha y hora exacta de la generación del informe.
    - Requisito de calidad. El script no puede ser un bloque de código incomprensible. Debe estar **debidamente documentado** mediante **comentarios (#)** explicando qué hace exactamente cada variable, tubería (|) y comando utilizado.

      generamos el archivo que se encuentra en el repositorio y lo ejecutamos

      no he conseguido generar correctamente el script en ubuntu por falta de organizacion y gestion de tiempo sin embargo esta subido correctamente a github.

Reto B: Configuración del "Escudo" Perimetral con UFW

Bajo el capó de Ubuntu trabaja el potente motor de filtrado de paquetes *iptables* (o *nftables*). Sin embargo, nosotros utilizaremos **UFW (Uncomplicated Firewall)**, una interfaz amigable que nos facilita la vida. Vamos a aplicar la arquitectura más segura: una política de "Denegación por Defecto". Todo lo que no esté explícitamente invitado a entrar, se quedará fuera.

1. **Reglas de Sigilo** (*Drop* vs *Reject*). Configura el firewall para que el servidor ignore completamente las peticiones ICMP Echo Request (el famoso comando ping). Un servidor que hace drop (dejar caer el paquete sin responder) a los pings, se vuelve prácticamente invisible frente a los escaneos de red superficiales que buscan hosts vivos.
2. **Reglas de Servicio**. Por otro lado, si bloqueamos todo, nosotros tampoco podremos administrarlo. Permitid el acceso TCP exclusivamente al puerto 22 (SSH). Si, como buena práctica, habéis cambiado el puerto SSH por defecto a uno superior, aseguraos de abrir ese puerto específico.

```
daw1@daw1:~/scripts$ chmod +x scripts/check_intruders.sh
chmod: cannot access 'scripts/check_intruders.sh': No such file
daw1@daw1:~/scripts$ cd ..
daw1@daw1:~$ cd ..
daw1@daw1:/home$ cd daw1/
daw1@daw1:~$ chmod +x scripts/check_intruders.sh
daw1@daw1:~$ sudo ufw default deny incoming
[sudo] password for daw1:
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
daw1@daw1:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
daw1@daw1:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
```

```
# ok icmp code for FORWARD
-A ufw-before-forward -p icmp --icmp-type destination-unreachable -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type time-exceeded -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type parameter-problem -j ACCEPT
-A ufw-before-forward -p icmp --icmp-type echo-request -j DROP
```

3. **Activación y Evidencias**. Ejecuta *sudo ufw enable*. Posteriormente, verifica el estado y la carga real de las reglas con los comandos *sudo ufw status numbered* y *sudo ufw status verbose*.

```
 if BROADCAST, RETURN
A ufw-not-local -m addrtype --dst-type BROADCAST -j R


aw1@daw1:~$ sudo ufw enable
irewall is active and enabled on system startup
aw1@daw1:~$
```

4. **Prueba cruzada**. En vuestra documentación, debéis incluir capturas demostrando que desde la máquina anfitriona el *ping* da error de tiempo de espera, pero la conexión por cliente SSH (como Windows PowerShell o el Terminal) sí se establece correctamente.

```
    inet6 fe80::ecee:eeff:feee:eeee/64 scope link
       valid_lft forever preferred_lft forever
daw1@daw1:~$ ping 192.168.20.33
PING 192.168.20.33 (192.168.20.33) 56(84) bytes of data.
64 bytes from 192.168.20.33: icmp_seq=1 ttl=64 time=0.057 ms
```

```
S C:\Users\DAW1> ssh DAW1@192.168.20.33
```